

ARGO CD - TASK

1. Setup ARGO CD

1. Prerequisites

You need:

- Kubernetes cluster running
- kubectl
- Working kubeconfig
- Cluster DNS/CoreDNS
- Internet access from the machine/cluster as required

The official getting-started guide lists kubectl, a valid kubeconfig, and CoreDNS as prerequisites.

Check your cluster

kubectl get nodes

```
MINGW64:/c:/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-10-0-16-137.ec2.internal        Ready    <none>   42h    v1.36.4-eks-3b4a6ca
ip-10-0-19-135.ec2.internal        Ready    <none>   42h    v1.36.4-eks-3b4a6ca
ip-10-0-4-225.ec2.internal         Ready    <none>   42h    v1.36.4-eks-3b4a6ca
ip-10-0-8-205.ec2.internal         Ready    <none>   42h    v1.36.4-eks-3b4a6ca

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n kube-system
NAME                                READY    STATUS    RESTARTS    AGE
aws-node-5lg57                      2/2     Running   0            42h
aws-node-nzgxf                      2/2     Running   0            42h
aws-node-rn1xn                      2/2     Running   0            42h
aws-node-wkpsp                      2/2     Running   0            42h
coredns-79f8dffc57-w2lvz            1/1     Running   0            42h
coredns-79f8dffc57-wm4hb            1/1     Running   0            42h
eks-pod-identity-agent-54v6z        1/1     Running   0            42h
eks-pod-identity-agent-9djbd        1/1     Running   0            42h
eks-pod-identity-agent-fflsj        1/1     Running   0            42h
eks-pod-identity-agent-p5mvp        1/1     Running   0            42h
kube-proxy-k8ljz                    1/1     Running   0            42h
kube-proxy-mkvz7                    1/1     Running   0            42h
kube-proxy-rvdx9                    1/1     Running   0            42h
kube-proxy-tchpf                    1/1     Running   0            42h
metrics-server-6877797457-rhn56     1/1     Running   0            42h

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

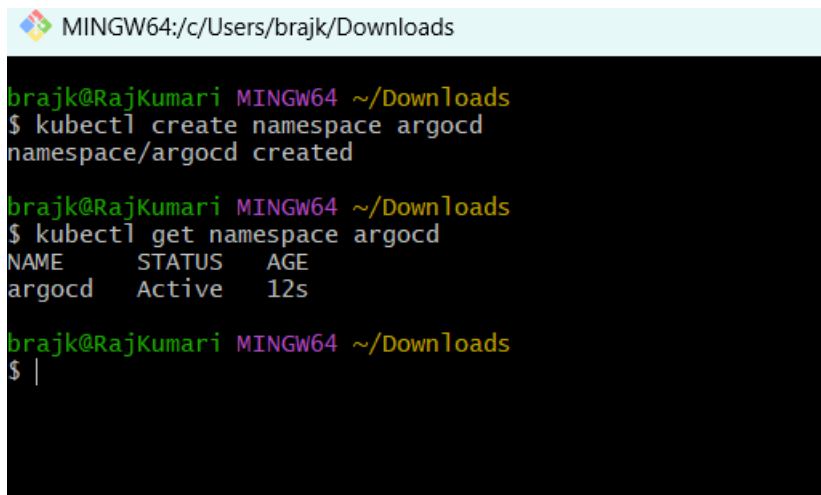
2. Create the Argo CD namespace

Create a dedicated namespace:

```
kubectl create namespace argocd
```

Verify:

```
kubectl get namespace argocd
```

A terminal window with a light blue title bar showing the path 'MINGW64:/c/Users/brajk/Downloads'. The terminal has a black background with green and yellow text. It shows the user 'brajk@RajKumari' in a 'MINGW64' shell at '~/Downloads'. The first command is '\$ kubectl create namespace argocd', followed by the output 'namespace/argocd created'. The second command is '\$ kubectl get namespace argocd', followed by a table of namespace details. The third command is '\$ kubectl get namespace argocd' again, followed by a cursor.

```
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl create namespace argocd
namespace/argocd created

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get namespace argocd
NAME      STATUS   AGE
argocd    Active   12s

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

3. Install Argo CD

For a learning environment, use the official stable installation manifest:

```
kubectl apply -n argocd --server-side --force-conflicts \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

This installs Argo CD into the argocd namespace. The official documentation specifically uses server-side apply because some Argo CD CRDs can exceed the client-side annotation size limit.

Check the resources

```
kubectl get all -n argocd
```

You should eventually see components such as:

pod/argocd-application-controller-...

pod/argocd-applicationset-controller-...

pod/argocd-dex-server-...

pod/argocd-notifications-controller-...

pod/argocd-redis-...

pod/argocd-repo-server-...

pod/argocd-server-...

Check specifically:

kubectl get pods -n argocd

Wait until the pods become:

Running

For example:

NAME	READY	STATUS	RESTARTS
argocd-application-controller-...	1/1	Running	0
argocd-applicationset-controller-...	1/1	Running	0
argocd-dex-server-...	1/1	Running	0
argocd-redis-...	1/1	Running	0
argocd-repo-server-...	1/1	Running	0
argocd-server-...	1/1	Running	0

```
brajk@RajKumari MINGW64 ~/Downloads
```

```
$ kubectl create namespace argocd
namespace/argocd created
```

```
brajk@RajKumari MINGW64 ~/Downloads
```

```
$ kubectl get namespace argocd
```

NAME	STATUS	AGE
argocd	Active	12s

```
brajk@RajKumari MINGW64 ~/Downloads
```

```
$ kubectl apply -n argocd --server-side --force-conflicts \
> -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install
.yaml
customresourcedefinition.apiextensions.k8s.io/applications.argoproj.io serversid
e-applied
customresourcedefinition.apiextensions.k8s.io/applicationsets.argoproj.io server
side-applied
customresourcedefinition.apiextensions.k8s.io/appprojects.argoproj.io serverside
-applied
serviceaccount/argocd-application-controller serverside-applied
serviceaccount/argocd-applicationset-controller serverside-applied
serviceaccount/argocd-dex-server serverside-applied
serviceaccount/argocd-notifications-controller serverside-applied
serviceaccount/argocd-redis serverside-applied
serviceaccount/argocd-repo-server serverside-applied
serviceaccount/argocd-server serverside-applied
role.rbac.authorization.k8s.io/argocd-application-controller serverside-applied
role.rbac.authorization.k8s.io/argocd-applicationset-controller serverside-appli
ed
role.rbac.authorization.k8s.io/argocd-dex-server serverside-applied
role.rbac.authorization.k8s.io/argocd-notifications-controller serverside-applie
d
role.rbac.authorization.k8s.io/argocd-redis serverside-applied
role.rbac.authorization.k8s.io/argocd-server serverside-applied
clusterrole.rbac.authorization.k8s.io/argocd-application-controller serverside-a
pplied
clusterrole.rbac.authorization.k8s.io/argocd-applicationset-controller serversid
e-applied
clusterrole.rbac.authorization.k8s.io/argocd-server serverside-applied
rolebinding.rbac.authorization.k8s.io/argocd-application-controller serverside-a
pplied
rolebinding.rbac.authorization.k8s.io/argocd-applicationset-controller serversid
e-applied
rolebinding.rbac.authorization.k8s.io/argocd-dex-server serverside-applied
rolebinding.rbac.authorization.k8s.io/argocd-notifications-controller serverside
-applied
rolebinding.rbac.authorization.k8s.io/argocd-redis serverside-applied
rolebinding.rbac.authorization.k8s.io/argocd-server serverside-applied
clusterrolebinding.rbac.authorization.k8s.io/argocd-application-controller serve
rside-applied
clusterrolebinding.rbac.authorization.k8s.io/argocd-applicationset-controller se
rverside-applied
clusterrolebinding.rbac.authorization.k8s.io/argocd-server serverside-applied
configmap/argocd-cm serverside-applied
configmap/argocd-cmd-params-cm serverside-applied
configmap/argocd-gpg-keys-cm serverside-applied
configmap/argocd-notifications-cm serverside-applied
```

```

MINGW64/c/Users/brajk/Downloads
argocd-applicationset-controller-5d7d5896f-mvwz1 1/1 Running 5 (3m54s ago) 15m
argocd-dex-server-674d8c59ff-fxgkj 1/1 Running 2 (15m ago) 15m
argocd-notifications-controller-564d78577b-jnsvs 1/1 Running 0 15m
argocd-redis-55d565584d-25gm7 1/1 Running 0 15m
argocd-repo-server-6995847c8-626mg 1/1 Running 0 15m
argocd-server-79c5c9ccc8-xqk94 1/1 Running 0 15m

brajk@Rajkumari MINGW64 ~/Downloads
$ kubectl get all -n argocd
NAME READY STATUS RESTARTS AGE
pod/argocd-application-controller-0 1/1 Running 0 15m
pod/argocd-applicationset-controller-5d7d5896f-mvwz1 1/1 Running 5 (4m5s ago) 15m
pod/argocd-dex-server-674d8c59ff-fxgkj 1/1 Running 2 (15m ago) 15m
pod/argocd-notifications-controller-564d78577b-jnsvs 1/1 Running 0 15m
pod/argocd-redis-55d565584d-25gm7 1/1 Running 0 15m
pod/argocd-repo-server-6995847c8-626mg 1/1 Running 0 15m
pod/argocd-server-79c5c9ccc8-xqk94 1/1 Running 0 15m

NAME TYPE CLUSTER-IP EXTERNAL-IP PO
RT(S) AGE
service/argocd-application-controller ClusterIP 10.100.245.217 <none> 70
00/TCP,8080/TCP 16m
service/argocd-dex-server ClusterIP 10.100.154.174 <none> 55
56/TCP,5557/TCP,5558/TCP 16m
service/argocd-metrics ClusterIP 10.100.69.39 <none> 80
82/TCP 16m
service/argocd-notifications-controller-metrics ClusterIP 10.100.204.189 <none> 90
01/TCP 16m
service/argocd-redis ClusterIP 10.100.77.31 <none> 63
79/TCP 16m
service/argocd-repo-server ClusterIP 10.100.18.153 <none> 80
81/TCP,8084/TCP 16m
service/argocd-server ClusterIP 10.100.107.255 <none> 80
/TCP,443/TCP 16m
service/argocd-server-metrics ClusterIP 10.100.130.86 <none> 80
83/TCP 16m

NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/argocd-application-controller 1/1 1 1 15m
deployment.apps/argocd-dex-server 1/1 1 1 15m
deployment.apps/argocd-notifications-controller 1/1 1 1 15m
deployment.apps/argocd-redis 1/1 1 1 15m
deployment.apps/argocd-repo-server 1/1 1 1 15m
deployment.apps/argocd-server 1/1 1 1 15m

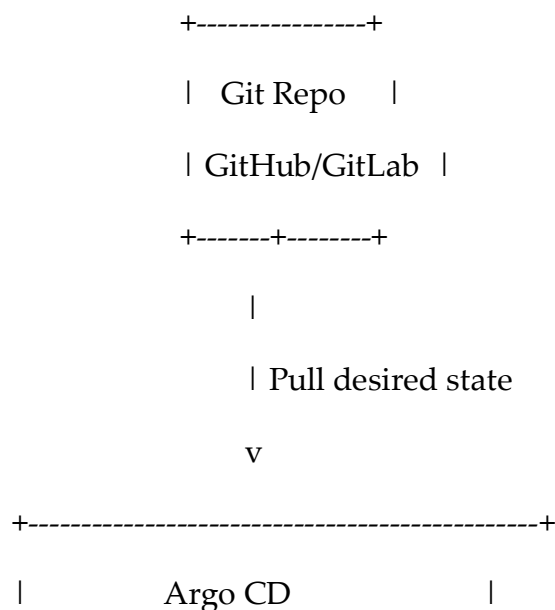
NAME DESIRED CURRENT READY AGE
replicaset.apps/argocd-application-controller-5d7d5896f 1 1 1 16m
replicaset.apps/argocd-dex-server-674d8c59ff 1 1 1 15m
replicaset.apps/argocd-notifications-controller-564d78577b 1 1 1 15m
replicaset.apps/argocd-redis-55d565584d 1 1 1 15m
replicaset.apps/argocd-repo-server-6995847c8 1 1 1 15m
replicaset.apps/argocd-server-79c5c9ccc8 1 1 1 15m

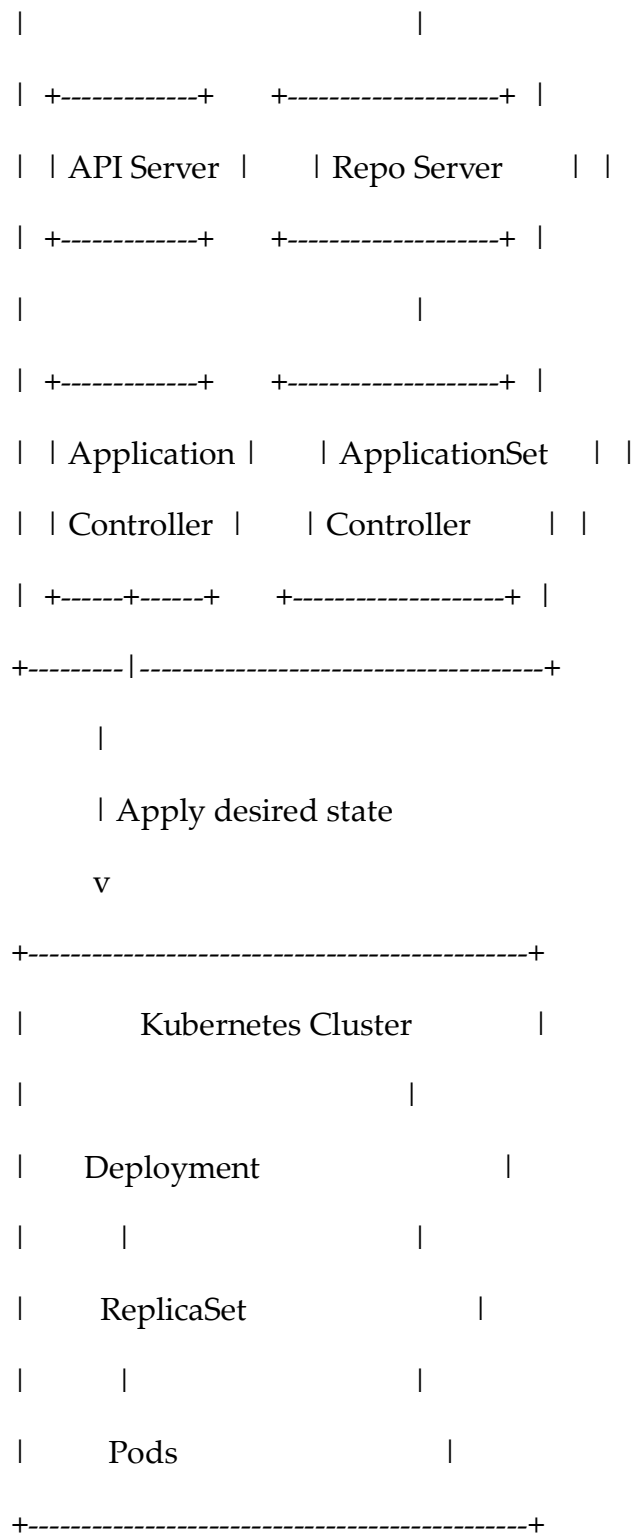
NAME READY AGE
statefulset.apps/argocd-application-controller 1/1 15m

```

4. Understand the Argo CD components

This is important for interviews.





Important components

argocd-server

Provides:

- Web UI

- API
- CLI access

argocd-repo-server

Responsible for retrieving and generating Kubernetes manifests from Git repositories and other supported sources.

argocd-application-controller

This is the most important component for GitOps reconciliation.

It compares:

Git desired state

vs

Kubernetes actual state

If they are different, Argo CD can synchronize the cluster.

argocd-redis

Used for Argo CD's internal caching/state-related operations.

ApplicationSet controller

Helps generate multiple Argo CD Applications from templates.

5. Check the Argo CD services

Run:

```
kubectl get svc -n argocd
```

```
MINGW64:/c/Users/brajK/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get svc -n argocd
```

NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
argocd-applicationset-controller	17m	ClusterIP	10.100.245.217	<none>	7000/TCP, 8080/TCP
argocd-dex-server	17m	ClusterIP	10.100.154.174	<none>	5556/TCP, 5557/TCP, 5558/TCP
argocd-metrics	17m	ClusterIP	10.100.69.39	<none>	8082/TCP
argocd-notifications-controller-metrics	17m	ClusterIP	10.100.204.189	<none>	9001/TCP
argocd-redis	17m	ClusterIP	10.100.77.31	<none>	6379/TCP
argocd-repo-server	17m	ClusterIP	10.100.18.153	<none>	8081/TCP, 8084/TCP
argocd-server	17m	ClusterIP	10.100.107.255	<none>	80/TCP, 443/TCP
argocd-server-metrics	17m	ClusterIP	10.100.130.86	<none>	8083/TCP

6. Easiest method — Port Forward

For your learning lab, I recommend **port forwarding first**.

Run:

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

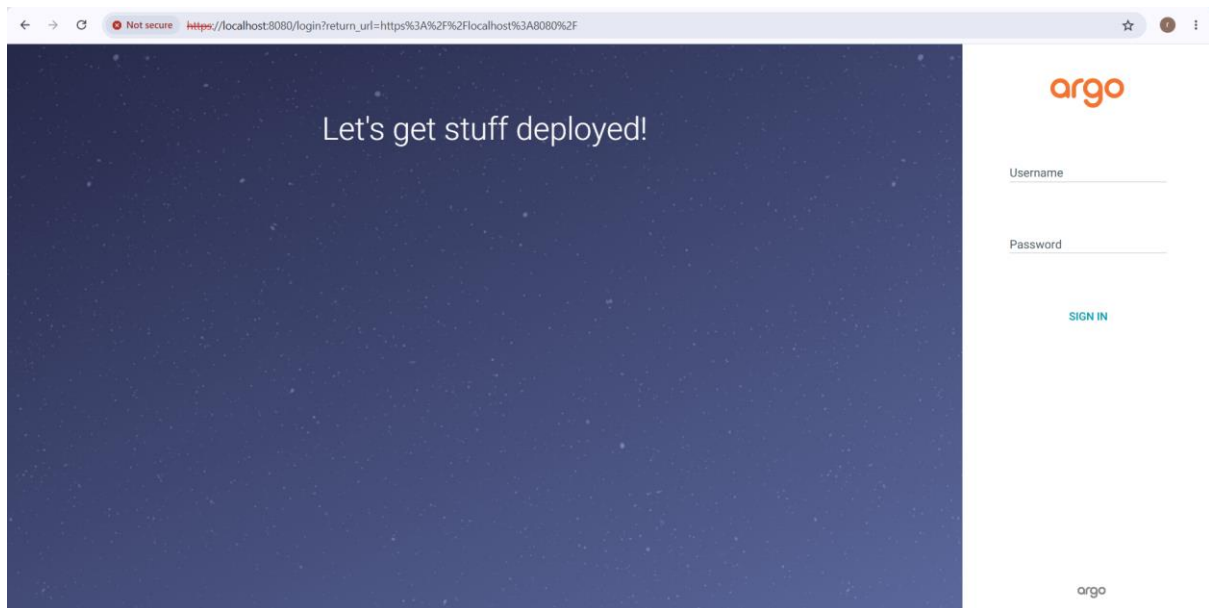
```
MINGW64:/c/Users/brajK/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl port-forward svc/argocd-server -n argocd 8080:443
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

Keep this terminal running.

Now open:

<https://localhost:8080>

You will probably get a browser certificate warning because the default Argo CD installation uses a self-signed certificate. This is expected for a lab



7. Get the initial Argo CD password

Open another terminal.

Run: `argocd admin initial-password -n argocd`

It will return something similar to:

Argo CD initial admin password:

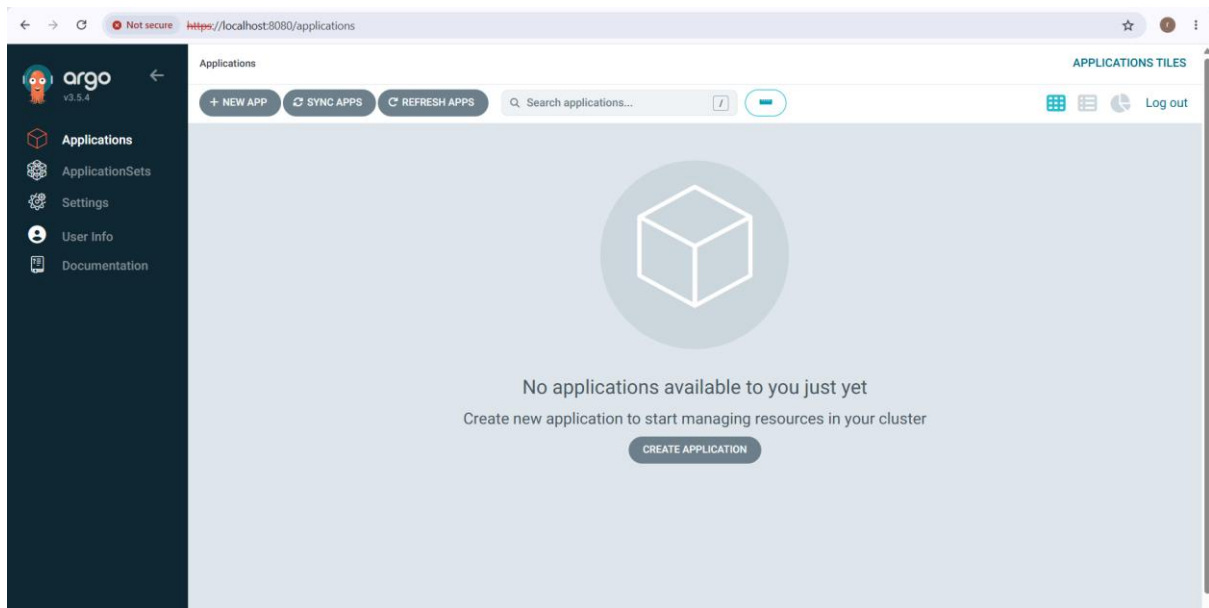
xx

The initial password is stored in the `argocd-initial-admin-secret`. The official documentation recommends deleting that initial secret after changing the password.

You can also retrieve it directly:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
```

```
-o jsonpath="{.data.password}" | base64 -d
```



8. Login to Argo CD UI

Open:

<https://localhost:8080>

Login:

Username:

admin

Password:

<password obtained above>

You should now see the Argo CD dashboard.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> winget search argocd
Name Id Version Match Source
-----
ArgoCD argoproj.argocd 3.5.3 winget
Radar skyhook-io.Radar 1.15.0 Tag: argocd winget
PS C:\Windows\system32> winget install --id argoproj.argocd
Found ArgoCD [argoproj.argocd] Version 3.5.3
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/argoproj/argo-cd/releases/download/v3.5.3/argocd-windows-amd64.exe
240 MB / 240 MB
Successfully verified installer hash
Starting package install...
Command line alias added: "argocd"
Successfully installed
PS C:\Windows\system32>
```

```
MINGW64:/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ argocd admin initial-password -n argocd
bash: argocd: command not found

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl -n argocd get secret argocd-initial-admin-secret \
> -o jsonpath="{.data.password}" | base64 -d
G7ti7SIRPxnLnssX
brajk@RajKumari MINGW64 ~/Downloads
$ AC

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

TASK 1.5 — Access Argo CD UI

For your lab, use port forwarding.

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Keep this terminal open.

Open browser:

<https://localhost:8080>

You may get a certificate warning. This is expected for the default lab installation.

TASK 1.6 — Get Argo CD password

Open another Git Bash terminal.

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d
```

Copy the password.

Login:

Username: admin

Password: <password>

TASK 1.7 — Login using CLI

```
argocd login localhost:8080 \
--username admin \
--password <PASSWORD> \
--insecure
```

Verify:

```
argocd account get-user-info
```

TASK 1.8 — Change initial password

Run:

```
argocd account update-password
```

Enter:

Current password

New password

Confirm password

Then delete the initial secret:

```
kubectl delete secret argocd-initial-admin-secret -n argocd
```

TASK 1.9 — Verify Task 1

Run:

```
kubectl get pods -n argocd
```

Then:

```
argocd version
```

Then:

```
argocd cluster list
```

You should see your Kubernetes cluster.

Task 1 complete

At this point:

EKS

|

+-- Argo CD

|

+-- Argo CD UI

|

+-- Argo CD CLI

```
MINGW64/c/Users/bradjk/Downloads
bradjk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS
argocd-application-controller-0     1/1     Running   0
argocd-applicationset-controller-76fd8cdd4f-kw9qz  1/1     Running   0
argocd-dex-server-66c78cf887-s2bbc  1/1     Running   0
argocd-notifications-controller-7fb9868fd6-wc99q  1/1     Running   0
argocd-redis-bdbdffc4-8jff6r       1/1     Running   0
argocd-repo-server-d89c7967d-p9fvh  1/1     Running   0
argocd-server-776b7cdd4d-l855p     1/1     Running   0

bradjk@RajKumari MINGW64 ~/Downloads
$ argocd version
argocd: v3.5.3+c9c369e
  BuildDate: 2026-09-14T07:38:20Z
  GitCommit: c9c369efcc5b2a0bd720803f8d14a1c3eaddf579
  GitTreeState: clean
  GoVersion: go1.26.4
  Compiler: gc
  Platform: windows/amd64
argocd-server: v3.5.4
  BuildDate: 2026-10-06T20:09:26Z
  GitCommit: d6d5b248ce0e1a2c512068002a93d3319767087
  GitTreeState: clean
  GitTag: v3.5.4
  GoVersion: go1.26.4
  Compiler: gc
  Platform: linux/amd64
  Customize Version: v5.8.1 2026-02-09T16:15:27Z
  Helm Version: v4.2.1-gd591a19
  Kubectl Version: v0.36.1
  Jsonnet Version: v0.22.0

bradjk@RajKumari MINGW64 ~/Downloads
$ argocd cluster list
SERVER          NAME      VERSION  STATUS  MESSAGE                                PROJECT
https://kubernetes.default.svc  in-cluster  1.37.0   Unknown  Cluster has no applications and is not being monitored.
```

TASK 2 — CREATE JENKINS CI JOB

Now we create the CI portion.

The purpose of this Jenkins job is:

GitHub

|

v

Checkout

|

v

Test

|

v

Validation

|

v

Success

It does **not** deploy yet.

Step 2.1 — Create GitHub application repository

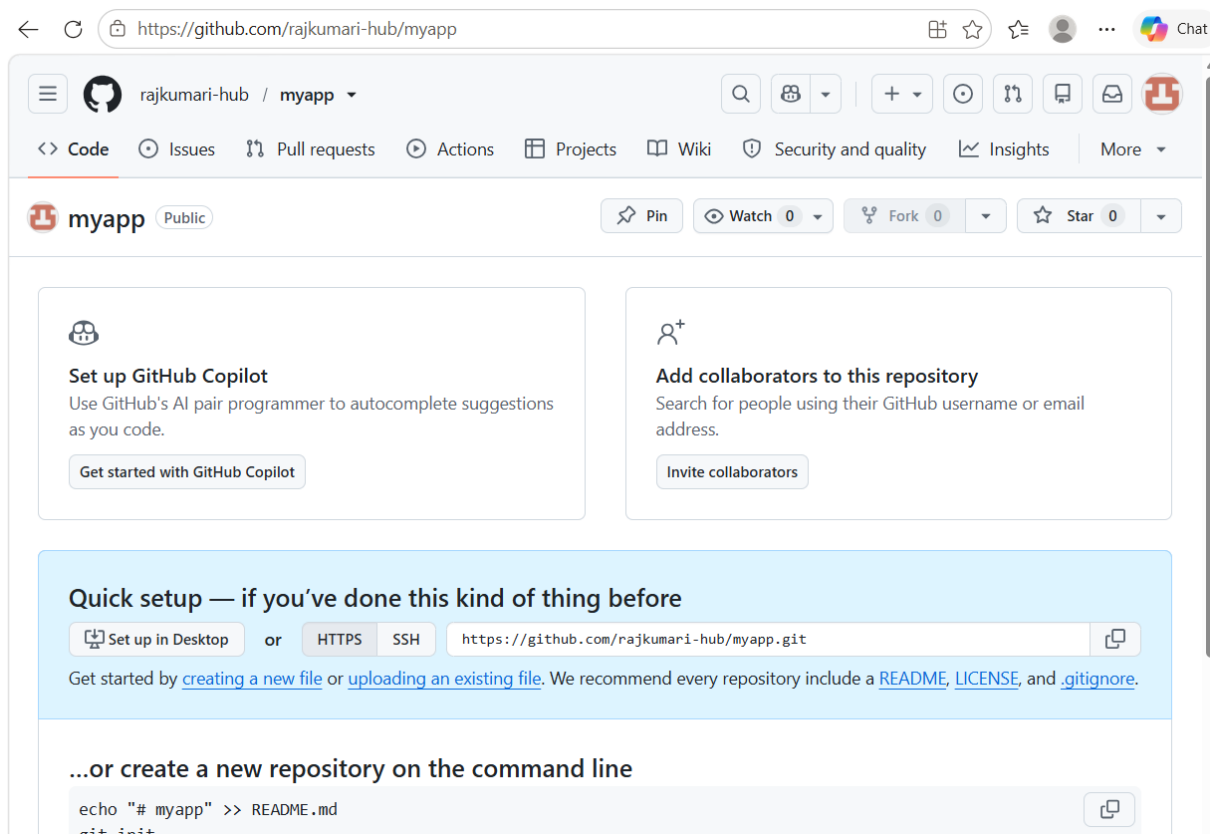
Create a GitHub repository:

myapp

Clone it:

```
git clone https://github.com/YOUR_USERNAME/myapp.git
```

```
cd myapp
```



Step 2.2 — Create application

Create:

index.html

```
MINGW64:/c/Users/braj/Downloads/myapp
<html>
<head>
  <title>DevOps Demo by Rajkumari</title>
</head>
<body>
  <h1>Hello from Jenkins + Docker + Argo CD + EKS</h1>
</body>
</html>
~
~
~
~
~
~
~
```

Step 2.3 — Create Dockerfile

Create:

Dockerfile

Contents:

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Your repository:

myapp/

├── Dockerfile

└── index.html

```
MINGW64:/c/Users/braj/Downloads/myapp
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
~
~
~
~
~
```

Step 2.4 — Push to GitHub

git add .

git commit -m "Initial application"

git push origin main

Verify the files are visible on GitHub.

```
MINGW64:/c/Users/braj/Downloads/myapp
xt time Git touches it
warning: in the working copy of 'index.html', LF will be replaced by CRLF the ne
xt time Git touches it

brajk@RajKumari MINGW64 ~/Downloads/myapp (main)
$ git commit -m "Initial application"
[main (root-commit) 19b0bcf] Initial application
 2 files changed, 13 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 index.html

brajk@RajKumari MINGW64 ~/Downloads/myapp (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 449 bytes | 449.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/rajkumari-hub/myapp.git
 * [new branch]      main -> main

brajk@RajKumari MINGW64 ~/Downloads/myapp (main)
$
```

Step 2.5 — Create Jenkins job

Open Jenkins.

Select:

New Item

Enter:

myapp-ci

Select:

Pipeline

Click:

OK

The screenshot shows the Jenkins dashboard. At the top, there's a header with the Jenkins logo, a search icon, a settings icon, and a user profile icon. Below the header, it says "No builds in the queue." and "Build Executor Status" with a dropdown menu showing "0/2". There's a button to "Add description". Below this, there's a filter section with "All" and a "+" button. The main part of the dashboard is a table of pipelines.

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	java-working-app	1 mo 0 days #1	N/A	26 sec
✓	☀	java-working-app-parameterized	1 mo 0 days #2	N/A	7.6 sec
⋮	☀	myapp-ci	N/A	N/A	N/A

Step 2.6 — Create Jenkins pipeline

Go to:

Pipeline

→ Definition

→ Pipeline script

Use:

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
stage('Checkout') {  
    steps {  
        git branch: 'main',  
            url: 'https://github.com/YOUR_USERNAME/myapp.git'  
    }  
}
```

```
stage('Validate') {  
    steps {  
        sh '''  
            echo "Checking files..."  
            test -f Dockerfile  
            test -f index.html  
        '''  
    }  
}
```

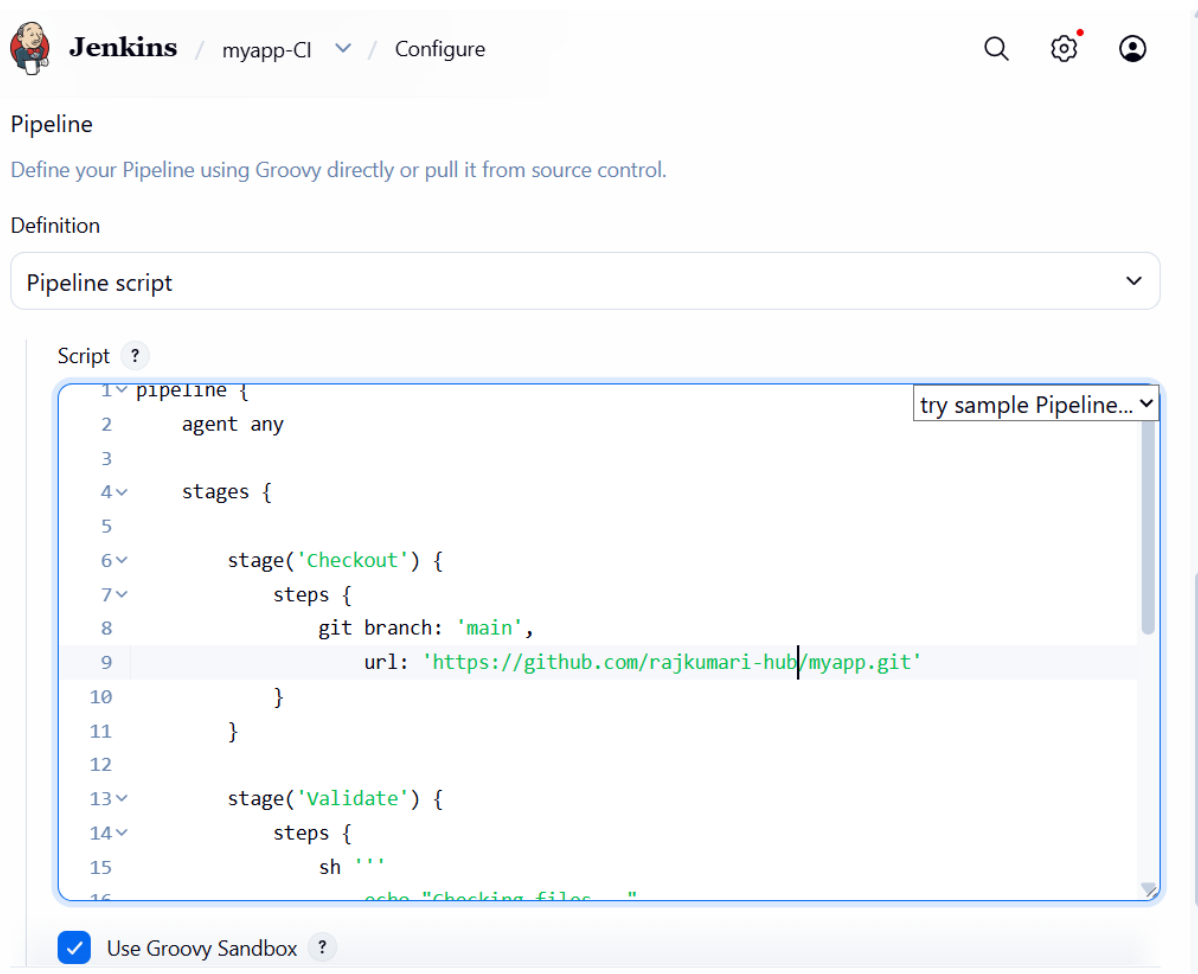
```
stage('Test') {  
    steps {  
        sh '''  
            echo "Running application tests..."  
            echo "Tests passed"  
        '''  
    }  
}
```

```
}  
  
}
```

Replace:

YOUR_USERNAME

with your GitHub username.



Step 2.7 — Run Jenkins CI

Click:

Build Now

Expected stages:

Checkout

✓

Validate

✓

Test

✓

Console output should end successfully.

The screenshot shows the Jenkins web interface in a browser. The address bar indicates a 'Not secure' connection to '54.81.194.251:8080/job/myapp-CI/'. The Jenkins logo and the pipeline name 'myapp-CI' are at the top. A sidebar on the left contains links: 'Delete Pipeline', 'Configure', 'Rename', 'Stages', and 'Pipeline Syntax'. The main content area shows a 'Builds' section with a search filter. Under the 'Today' heading, there is one build entry: a green checkmark icon, '#1', and '4:30 PM'. Below this, the pipeline name 'myapp-CI' is displayed with a green checkmark icon. To the right of the pipeline name is a button labeled 'Add c'.

Permalinks

- [Last build \(#1\), 12 min ago](#)
- [Last stable build \(#1\), 12 min ago](#)
- [Last successful build \(#1\), 12 min ago](#)
- [Last completed build \(#1\), 12 min ago](#)



The screenshot shows a web browser window with the Jenkins interface. The address bar indicates a 'Not secure' connection to '54.81.194.251:8080/job/myapp-cl/1/console'. The Jenkins header shows the logo, the job name 'myapp-cl', and the build number '#1'. The console output displays the following text:

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Validate)
[Pipeline] sh
+ echo 'Checking files...'
Checking files...
+ test -f Dockerfile
+ test -f index.html
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] sh
+ echo 'Running application tests...'
Running application tests...
+ echo 'Tests passed'
Tests passed
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

The final line, 'Finished: SUCCESS', is highlighted with a red rectangular box.

Step 2.8 — Understand Task 2

Jenkins has now proved:

GitHub



Jenkins



Checkout



Validate



Test

Task 2 complete.

TASK 3 — CREATE DOCKER IMAGE + PUSH TO ECR

Now Jenkins will build the Docker image.

Architecture:

GitHub



v

Jenkins

|

| docker build

v

Docker Image

|

| docker push

v

AWS ECR

Step 3.1 — Create ECR repository

Run from Git Bash:

```
aws ecr create-repository \  
--repository-name myapp \  
--region us-east-1
```

Verify:

```
aws ecr describe-repositories \  
--repository-names myapp \  
--region us-east-1
```

```

MINGW64:/c/Users/braj/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ aws ecr create-repository \
> --repository-name myapp \
> --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:820982384063:repository/myapp",
    "registryId": "820982384063",
    "repositoryName": "myapp",
    "repositoryUri": "820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp",
    "createdAt": "2026-10-07T22:02:57.347000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}

brajk@RajKumari MINGW64 ~/Downloads
$ aws ecr describe-repositories \
> --repository-names myapp \
> --region us-east-1
{
  "repositories": [
    {
      "repositoryArn": "arn:aws:ecr:us-east-1:820982384063:repository/myapp",
      "registryId": "820982384063",
      "repositoryName": "myapp",
      "repositoryUri": "820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp",
      "createdAt": "2026-10-07T22:02:57.347000+05:30",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
        "scanOnPush": false
      },
      "encryptionConfiguration": {
        "encryptionType": "AES256"
      }
    }
  ]
}

brajk@RajKumari MINGW64 ~/Downloads
$ |

```

Step 3.2 — Find ECR URL

Run:

```

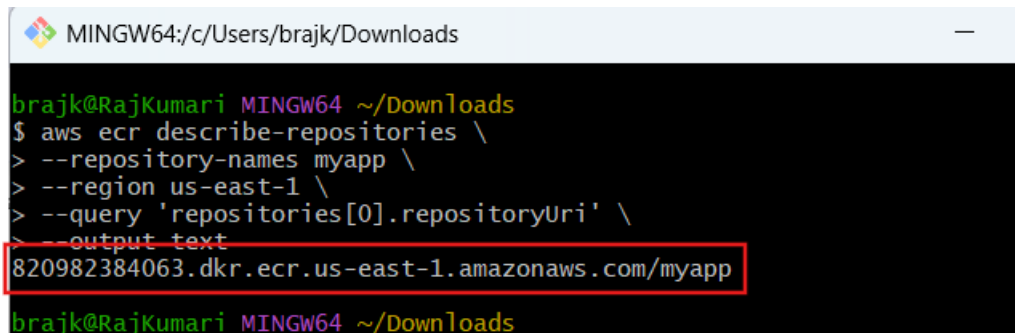
aws ecr describe-repositories \
--repository-names myapp \
--region us-east-1 \
--query 'repositories[0].repositoryUri' \
--output text

```

You'll get something like:

123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp

Save this.

A terminal window titled 'MINGW64:/c/Users/braj/Downloads' shows a command prompt session. The user 'brajk@RajKumari' is in the directory '~/Downloads'. They run the command 'aws ecr describe-repositories --repository-names myapp --region us-east-1 --query 'repositories[0].repositoryUri' --output text'. The output is '820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp', which is highlighted with a red rectangle. The prompt returns to 'brajk@RajKumari MINGW64 ~/Downloads'.

Step 3.3 — Jenkins AWS permissions

Your Jenkins server needs permission to push to ECR.

If Jenkins is running on EC2, preferably attach an IAM role to the Jenkins EC2 instance.

For a lab, you can use:

AmazonEC2ContainerRegistryPowerUser

For production, use a least-privilege custom policy.

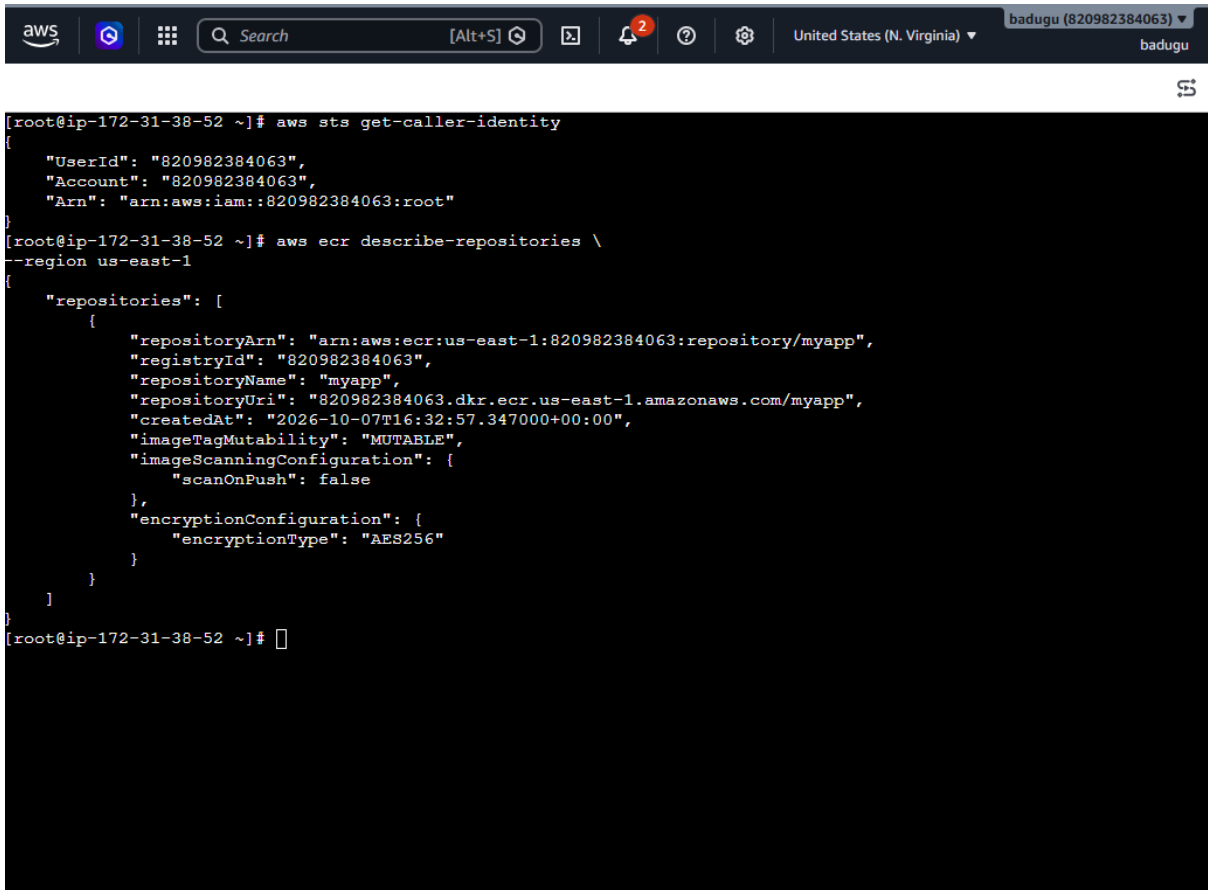
Test from the Jenkins machine:

```
aws sts get-caller-identity
```

Then:

```
aws ecr describe-repositories \
```

```
--region us-east-1
```



```
[root@ip-172-31-38-52 ~]# aws sts get-caller-identity
{
  "UserId": "820982384063",
  "Account": "820982384063",
  "Arn": "arn:aws:iam::820982384063:root"
}
[root@ip-172-31-38-52 ~]# aws ecr describe-repositories \
--region us-east-1
{
  "repositories": [
    {
      "repositoryArn": "arn:aws:ecr:us-east-1:820982384063:repository/myapp",
      "registryId": "820982384063",
      "repositoryName": "myapp",
      "repositoryUri": "820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp",
      "createdAt": "2026-10-07T16:32:57.347000+00:00",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
        "scanOnPush": false
      },
      "encryptionConfiguration": {
        "encryptionType": "AES256"
      }
    }
  ]
}
```

i-0ccbde82075284734 (Jenkin-server)

PublicIPs: 54.81.194.251 PrivateIPs: 172.31.38.52

Step 3.4 — Verify Docker on Jenkins

On the Jenkins machine:

```
docker --version
```

Then:

```
docker ps
```

If you receive:

permission denied

Jenkins needs permission to access Docker.

For a Linux Jenkins server, commonly:

```
sudo usermod -aG docker jenkins
```

Then restart Jenkins:

```
sudo systemctl restart jenkins
```

Log out/in or restart the agent if necessary.

Verify again.

Step 3.5 — Create Jenkins Docker job

Create:

myapp-docker-build

Select:

Pipeline

Step 3.6 — Jenkins Docker pipeline

Use:

```
pipeline {
```

```
    agent any
```

```
    environment {
```

```
        AWS_REGION = 'us-east-1'
```

```
        ECR_REPO = '820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp'
```

```
        IMAGE_TAG = "${BUILD_NUMBER}"
```

```
    }
```

```
    stages {
```

```
        stage('Checkout') {
```

```
            steps {
```

```
                git branch: 'main',
```

```
                url: 'https://github.com/rajkumari-hub/myapp.git'
```

```
    }  
  }  
  
  stage('Docker Build') {  
    steps {  
      sh '''  
        docker build \  
        -t ${ECR_REPO}:${IMAGE_TAG} .  
      '''  
    }  
  }  
}
```

```
stage('ECR Login') {  
  steps {  
    sh '''  
      aws ecr get-login-password \  
      --region ${AWS_REGION} | \  
      docker login \  
      --username AWS \  
      --password-stdin ${ECR_REPO}  
    '''  
  }  
}
```

```
stage('Push Image') {  
  steps {
```

```
sh ""  
    docker push ${ECR_REPO}:${IMAGE_TAG}  
    ""  
}  
}  
}
```

Change:

123456789012

to your AWS account ID.

← → ↺

⚠ Not secure 54.81.194.251:8080/job/myapp-docker-build/configure

 **Jenkins**

/ myapp-docker-build ▾ / Configure

🔍

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

3 agent any

4

5 environment {

6 AWS_REGION = 'us-east-1'

7 ECR_REPO = '820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp'

8 IMAGE_TAG = "\${BUILD_NUMBER}"

9 }

10

11 stages {

12

13 stage('Checkout') {

14 steps {

15 git branch: 'main',

16 url: 'https://github.com/rajkumari-hub/myapp.git'

17 }

18 }

try sample I

☒ Use Groovy Sandbox ?

Save

Apply

Step 3.7 — Run Jenkins

Click:

Build Now

Expected:

Checkout

✓

Docker Build

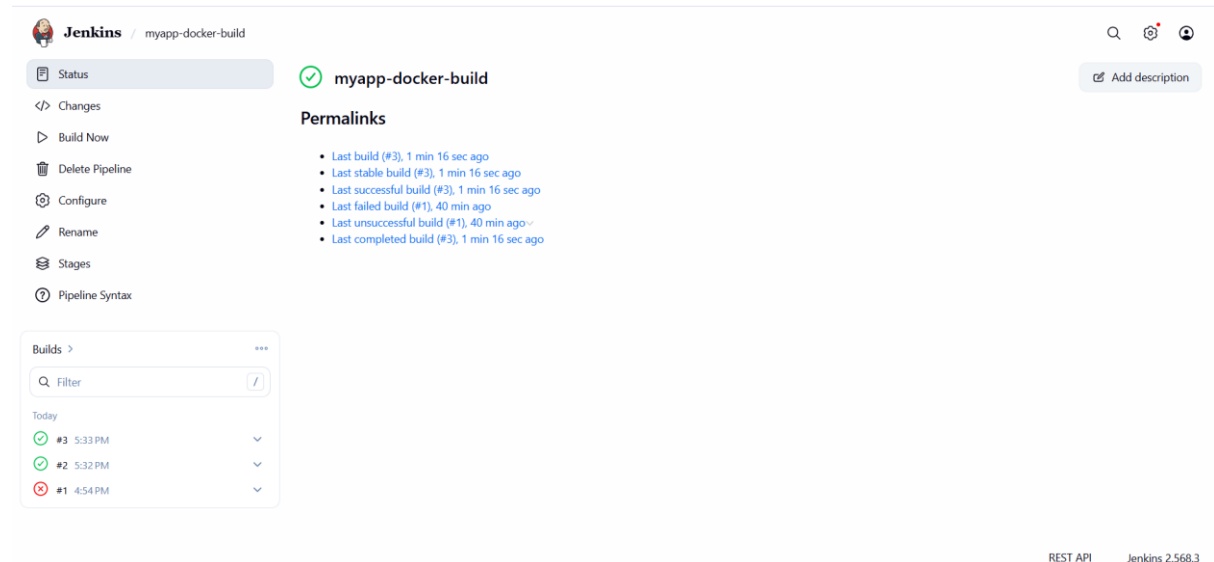
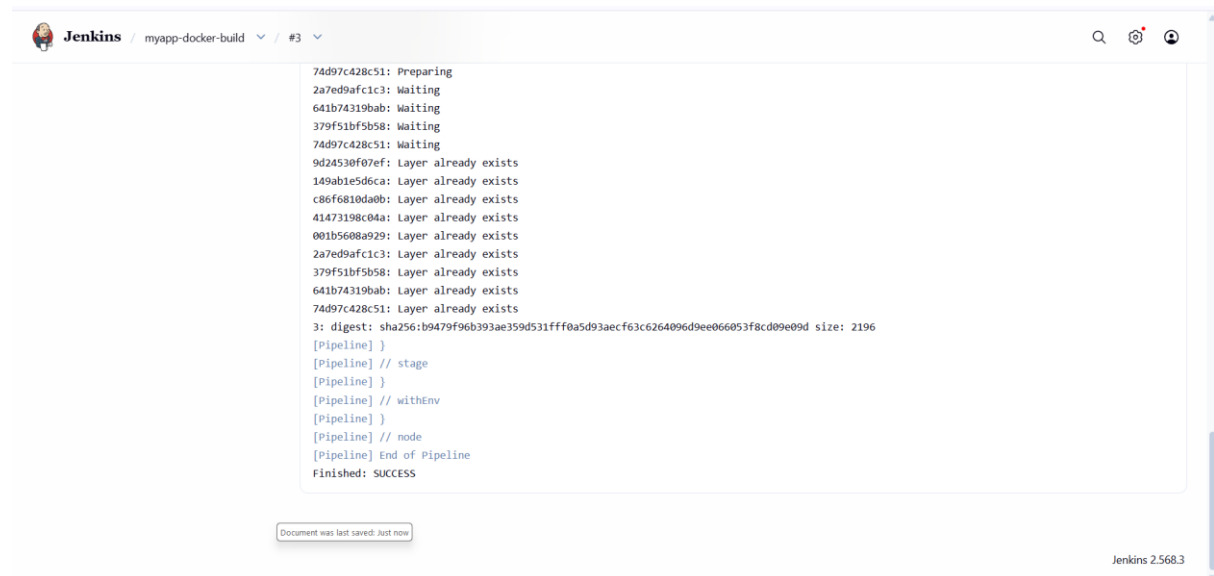
✓

ECR Login

✓

Push Image

✓



Step 3.8 — Verify ECR

Run:

aws ecr describe-images \

```
--repository-name myapp \
```

```
--region us-east-1
```

You should see an image tag such as:

1

or:

2

depending on the Jenkins build number.

Step 3.9 — Understand Task 3

You now have:

GitHub

|

v

Jenkins

|

| Docker build

v

myapp:1

|

| docker push

v

ECR

Task 3 complete.

```
MINGW64:/c/Users/brajK/Downloads/myapp
brajk@RajKumari MINGW64 ~/Downloads/myapp (main)
$ aws ecr describe-images \
> --repository-name myapp \
> --region us-east-1
{
  "imageDetails": [
    {
      "registryId": "820982384063",
      "repositoryName": "myapp",
      "imageDigest": "sha256:b9479f96b393ae359d531fff0a5d93aecf63c6264096d9ee066053f8cd09e09d",
      "imageTags": [
        "3",
        "2"
      ],
      "imageSizeInBytes": 26323590,
      "imagePushedAt": "2026-10-07T23:02:58.335000+05:30",
      "imageManifestMediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "artifactMediaType": "application/vnd.docker.container.image.v1+json",
      "imageStatus": "ACTIVE"
    }
  ]
}
```

TASK 4 – ARGO CD DEPLOYMENT TO EKS

Now we create the GitOps repository.

Create a second GitHub repository:

myapp-manifests

This repository contains **only Kubernetes deployment configuration**

Step 4.1 – Clone manifest repository

```
git clone https://github.com/YOUR_USERNAME/myapp-manifests.git
```

```
cd myapp-manifests
```

```
MINGW64:/c/Users/braj/Downloads/myapp-manifests  
brajk@RajKumari MINGW64 ~/Downloads  
$ git clone https://github.com/rajkumari-hub/myapp-manifests.git  
Cloning into 'myapp-manifests'...  
warning: You appear to have cloned an empty repository.  
  
brajk@RajKumari MINGW64 ~/Downloads  
$ cd myapp-manifests/  
  
brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)  
$ |
```

Step 4.2 — Create namespace.yaml

Create:

namespace.yaml

apiVersion: v1

kind: Namespace

metadata:

name: myapp

```
MINGW64:/c/Users/braj/Downloads/myapp-manifests  
apiVersion: v1  
kind: Namespace  
metadata:  
  name: myapp  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

Step 4.3 — Create deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: myapp

namespace: myapp

spec:

replicas: 2

selector:

matchLabels:

app: myapp

template:

metadata:

labels:

app: myapp

spec:

containers:

- name: myapp

image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:3 # Build
number should be match

ports:

- containerPort: 80

Replace:

123456789012

with your AWS account ID.

```

MINGW64:/c/Users/braj/Downloads/myapp-manifests
apiVersion: apps/v1
kind: Deployment

metadata:
  name: myapp
  namespace: myapp

spec:
  replicas: 2

  selector:
    matchLabels:
      app: myapp

  template:
    metadata:
      labels:
        app: myapp

    spec:
      containers:
        - name: myapp

          image: 820982384063.dkr.ecr.us-east-1.amazonaws.com/myapp:3

          ports:
            - containerPort: 80

```

Step 4.4 — Create service.yaml

apiVersion: v1

kind: Service

metadata:

name: myapp

namespace: myapp

spec:

type: LoadBalancer

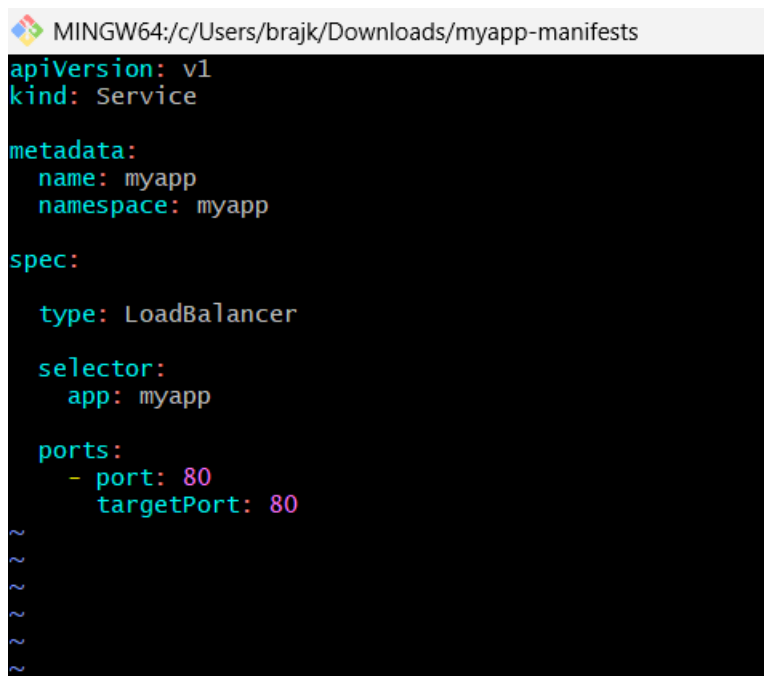
selector:

app: myapp

ports:

- port: 80

targetPort: 80

A terminal window with a black background and a light gray title bar. The title bar text is "MINGW64:/c/Users/brajik/Downloads/myapp-manifests". The terminal content shows a Kubernetes manifest file for a Service. The text is as follows:

```
apiVersion: v1
kind: Service

metadata:
  name: myapp
  namespace: myapp

spec:
  type: LoadBalancer

  selector:
    app: myapp

  ports:
    - port: 80
      targetPort: 80
```

At the bottom of the terminal, there are several tilde (~) characters on separate lines.

Step 4.5 — Commit manifest files

Your repository:

myapp-manifests/

├── namespace.yaml

├── deployment.yaml

└── service.yaml

Run:

git add .

git commit -m "Add Kubernetes manifests"

git push origin main

```
MINGW64:/c/Users/braj/Downloads/myapp-manifests

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ vi deployment.yaml

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ vi service.yaml

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ git add .
warning: in the working copy of 'deployment.yaml', LF will be replaced by CRLF t
he next time Git touches it
warning: in the working copy of 'namespace.yaml', LF will be replaced by CRLF t
he next time Git touches it
warning: in the working copy of 'service.yaml', LF will be replaced by CRLF th
e next time Git touches it

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ git commit -m "Added kubernetes manifests"
[main (root-commit) 40f83f4] Added kubernetes manifests
 3 files changed, 50 insertions(+)
 create mode 100644 deployment.yaml
 create mode 100644 namespace.yaml
 create mode 100644 service.yaml

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 695 bytes | 695.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/rajkumari-hub/myapp-manifests.git
 * [new branch]      main -> main

brajk@RajKumari MINGW64 ~/Downloads/myapp-manifests (main)
$ |
```

Step 4.6 — Create Argo CD Application

Now tell Argo CD:

Watch this Git repository and deploy it to my EKS cluster.

Create:

application.yaml

```
MINGW64:/c/Users/braj/Downloads/myapp-manifests
apiVersion: argoproj.io/v1alpha1
kind: Application

metadata:
  name: myapp
  namespace: argocd

spec:
  project: default

  source:
    repoURL: https://github.com/rajkumari-hub/myapp-manifests.git
    targetRevision: main
    path: .

  destination:
    server: https://kubernetes.default.svc
    namespace: myapp

  syncPolicy:
    automated:
      prune: true
      selfHeal: true

    syncOptions:
      - CreateNamespace=true
~
~
~
~
~
~
~
~
~
```

Replace:

YOUR_USERNAME : rajkumari-hub

Step 4.7 — Create Argo CD Application

Run:

```
kubectl apply -f application.yaml
```

Verify:

```
kubectl get applications -n argocd
```

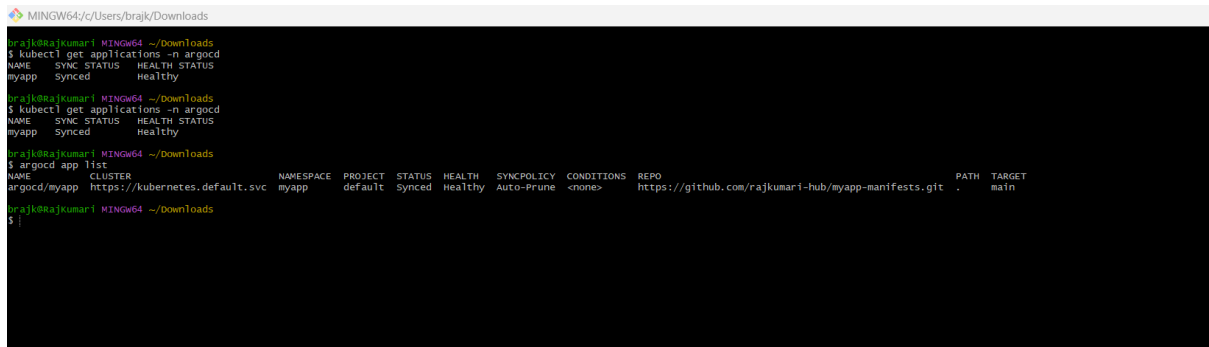
Expected:

NAME	SYNC STATUS	HEALTH STATUS
------	-------------	---------------

myapp	Synced	Healthy
-------	--------	---------

Also:

argocd app list



```
MINGW64/c:/Users/brak/Downloads
brak@rajakumar1 MINGW64 ~/Downloads
$ kubectl get applications -n argocd
NAME SYNC STATUS HEALTH STATUS
myapp Synced Healthy

brak@rajakumar1 MINGW64 ~/Downloads
$ kubectl get applications -n argocd
NAME SYNC STATUS HEALTH STATUS
myapp Synced Healthy

brak@rajakumar1 MINGW64 ~/Downloads
$ argocd app list
NAME CLUSTER
argocd/myapp https://kubernetes.default.svc

brak@rajakumar1 MINGW64 ~/Downloads
$ kubectl get applications -n argocd
NAME SYNC STATUS HEALTH STATUS
myapp Synced Healthy
```

Step 4.8 — Verify EKS

kubectl get pods -n myapp

Expected:

NAME	READY	STATUS
------	-------	--------

myapp-xxxxxxxxxx	1/1	Running
------------------	-----	---------

myapp-xxxxxxxxxx	1/1	Running
------------------	-----	---------

Check service:

kubectl get svc -n myapp

Expected:

NAME	TYPE	EXTERNAL-IP
------	------	-------------

myapp	LoadBalancer	...
-------	--------------	-----

```
MINGW64:/c/Users/brajK/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n myapp
NAME                                READY   STATUS    RESTARTS   AGE
myapp-5764db897b-gwtxf             1/1     Running   0           16m
myapp-5764db897b-lvcdv             1/1     Running   0           16m

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n myapp
NAME                                READY   STATUS    RESTARTS   AGE
myapp-5764db897b-gwtxf             1/1     Running   0           17m
myapp-5764db897b-lvcdv             1/1     Running   0           17m

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n myapp -o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: Pod
  metadata:
    creationTimestamp: "2026-10-07T18:20:18Z"
    generateName: myapp-5764db897b-
    generation: 1
    labels:
      app: myapp
      pod-template-hash: 5764db897b
      topology.kubernetes.io/region: us-east-1
```

Step 4.9 — Test application

Get the LoadBalancer hostname:

```
kubectl get svc myapp -n myapp
```

Wait until EXTERNAL-IP has a value.

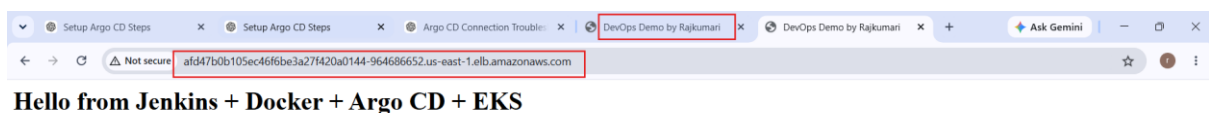
Then open:

<http://<EXTERNAL-IP>>

You should see:

Hello from Jenkins + Docker + Argo CD + EKS

Task 4 complete.





TASK 5 — CONNECT JENKINS → GIT → ARGO CD

Now we make the entire system automatic.

This is the most important part.

Suppose Jenkins creates:

myapp:5

and pushes:

ECR

|

+-- myapp:5

But Git currently says:

image: ../myapp:1

Argo CD doesn't automatically know about myapp:5.

So Jenkins must update:

myapp:1

to:

myapp:5

inside the GitOps repository.

Then Argo CD detects the Git change.

Step 5.1 — Complete pipeline

The final flow becomes:

Developer

|

| git push

v

GitHub Application Repo

|

v

Jenkins

|

+---- Checkout

|

+---- Test

|

+---- Docker Build

|

+---- Push ECR

|

+---- Update GitOps repo

|

v

GitHub Manifest Repo

|

v

Argo CD

|

v

EKS

|
v
Pods

Step 5.2 — Update Jenkins pipeline

Add this stage after Docker Push:

```
stage('Update GitOps Repository') {
```

```
    steps {
```

```
        sh '''
```

```
            git clone https://github.com/YOUR_USERNAME/myapp-manifests.git
```

```
            cd myapp-manifests
```

```
            sed -i "s#myapp:[0-9]*#myapp:${BUILD_NUMBER}#g" deployment.yaml
```

```
            git config user.email "jenkins@example.com"
```

```
            git config user.name "Jenkins"
```

```
            git add deployment.yaml
```

```
            git commit -m "Update image to ${BUILD_NUMBER}"
```

```
            git push origin main
```

```
        '''
```

```
}  
  
}
```

Important: for a real GitHub repository, don't put your GitHub password/token directly in the Jenkinsfile. Configure a Jenkins credential and use it securely.

Step 5.3 — Run Jenkins

Suppose Jenkins build number is:

10

Jenkins creates:

myapp:10

Pushes:

ECR

└── myapp:10

Then updates Git:

image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:10

Then:

git commit

git push

Step 5.4 — Argo CD detects Git change

Argo CD sees:

Old:

myapp:9

Git changes to:

myapp:10

Therefore:

Desired state changed

|

v

Argo CD

|

v

OutOfSync

|

v

Automatic Sync

|

v

EKS

Because we configured:

syncPolicy:

automated:

prune: true

selfHeal: true

Argo CD automatically synchronizes.

Step 5.5 — Verify rollout

Run:

```
kubectl get pods -n myapp
```

Then:

```
kubectl get deployment myapp -n myapp
```

Check the image:

```
kubectl get deployment myapp -n myapp \  
-o jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected:

123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:10

Check Argo CD:

```
argocd app get myapp
```

You want:

Sync Status: Synced

Health Status: Healthy

TASK 6 — TEST SELF-HEALING

This is an excellent interview/demo scenario.

First:

```
kubectl get deployment myapp -n myapp
```

Suppose:

replicas = 2

Now manually change Kubernetes:

```
kubectl scale deployment myapp \  
--replicas=5 \  
-n myapp
```

Check:

```
kubectl get deployment myapp -n myapp
```

Kubernetes temporarily has:

5 replicas

But Git says:

2 replicas

Argo CD detects:

Live State != Desired State

Because:

selfHeal: true

Argo CD changes it back to:

2 replicas

Check:

kubectl get deployment myapp -n myapp

This demonstrates **GitOps self-healing**.

TASK 7 – Test Git-based deployment

Now change:

replicas: 2

to:

replicas: 3

Commit:

git add deployment.yaml

git commit -m "Scale application to 3 replicas"

git push origin main

Argo CD detects:

Git:

replicas = 3

EKS:

replicas = 2

Argo CD synchronizes.

Verify:

```
kubectl get deployment myapp -n myapp
```

Expected:

READY

3/3

FINAL PROJECT — ALL TASKS

You can now explain your project like this:

Task 1 — Argo CD

EKS

|

+-- argocd namespace

|

+-- Argo CD

|

+-- GitOps Application

Task 2 — Jenkins CI

GitHub

↓

Jenkins

↓

Checkout

↓

Test

↓

Validate

Task 3 — Docker + ECR

Jenkins



Docker Build



Docker Image



ECR

Task 4 — Argo CD Deployment

GitOps Repository



Argo CD



EKS



Pods

Task 5 — Complete automation

Developer



GitHub



Jenkins



Test



Docker Build



ECR



Update GitOps Repository



Argo CD



EKS



Kubernetes Deployment



Pods

The most important distinction

CI

CD



v

v

Jenkins

Argo CD



Build/Test/Image

GitOps/Deploy



v

v

ECR

EKS

Jenkins = Continuous Integration

Argo CD = Continuous Delivery / GitOps

And the key production concept is:

Jenkins should not directly run `kubectl apply` to deploy the application in this GitOps model. Jenkins builds and publishes the artifact, updates the GitOps repository, and Argo CD reconciles that Git state into EKS.

The top screenshot shows the Argo CD web interface for the application 'myapp'. The interface includes a sidebar with navigation links (Applications, ApplicationSets, Settings, User Info, Documentation) and a main content area. The main content area displays the application's health status as 'Healthy' and its sync status as 'Synced to main(5f3d439)'. A 'LAST SYNC' section indicates a successful sync at 12 minutes ago. Below this, a diagram illustrates the application's resources: a 'myapp' service, a 'myapp' deployment, and two 'myapp-78f714999' pods. The bottom screenshot shows the 'Deployment History' for 'myapp'. It lists two deployments: one at 13 minutes ago (revision 5f3d439) and another at 41 minutes ago (revision bde8318). Each deployment entry includes details such as 'Deployed At', 'Revision', 'Authorized by', 'Time to deploy', 'Initiated by', and 'Active for'. The 'Source Parameters' section for each deployment points to the GitHub repository 'https://github.com/rajkumari-hub/myapp-manifests.git'.

argo v3.5.4

Applications

ApplicationSets

Settings

User Info

Documentation

Resource filters

NAME

KINDS

SYNC STATUS

DETAILS

APP HEALTH

HEALTHY

SUMMARY

PARAMETERS

MANIFEST

EVENTS

```
1 project: default
2 source:
3   repoURL: https://github.com/rajkumari-hub/myapp-manifests.git
4   path:
5   targetRevision: main
6 destination:
7   server: https://kubernetes.default.svc
8   namespace: myapp
9 syncPolicy:
10   automated:
11     prune: true
12     selfHeal: true
13   syncOptions:
14     - CreateNamespace=true
15
```

EDIT

argo v3.5.4

Applications

ApplicationSets

Settings

User Info

Documentation

Resource filters

NAME

KINDS

SYNC STATUS

HEALTH STATUS

DETAILS

APP HEALTH

HEALTHY

SUMMARY

PARAMETERS

MANIFEST

EVENTS

REASON	MESSAGE	COUNT	FIRST OCCURRED	LAST OCCURRED
ResourceUpdated	Updated health status: Progressing -> Healthy	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
OperationCompleted	Sync operation to 5f3d4392438330d5063daf348906eea9b4ea3529 succeeded	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
OperationStarted	Initiated automated sync to '5f3d4392438330d5063daf348906eea9b4ea3529'	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
ResourceUpdated	Updated sync status: OutOfSync -> Synced	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
ResourceUpdated	Updated health status: Healthy -> Progressing	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
OperationCompleted	Partial sync operation to 5f3d4392438330d5063daf348906eea9b4ea3529 succeeded	1	14m ago Today at 12:18 AM	14m ago Today at 12:18 AM
OperationStarted	Initiated automated sync to '5f3d4392438330d5063daf348906eea9b4ea3529'	1	14m ago	14m ago

argo v3.5.4

Applications

ApplicationSets

Settings

User Info

Documentation

Resource filters

NAME

KINDS

SYNC STATUS

HEALTH STATUS

APPLICATIONS

myapp

APPLICATION DETAILS TREE

DETAILS

DIFF

SYNC

SYNC STATUS

HISTORY AND ROLLBACK

DELETE

REFRESH

Log out

APP HEALTH

HEALTHY

SYNC STATUS

Synced to main(5f3d439) [🔗](#)

Auto sync is enabled.

Author: rajkumari-brajkumari7675@gmail.com -

Comment: build number 4

LAST SYNC

Sync OK to 5f3d439 [🔗](#)

Succeeded 16 minutes ago (Thu Oct 08 2026 00:18:55 GMT+0530)

Author: rajkumari-brajkumari7675@gmail.com -

Comment: build number 4

myapp

myapp-78f7149999

myapp-5764db897b

ReplicaSet 2

Service 1

Namespace 1

Deployment 1

Pod 4

Synced 3

OutOfSync 0

Progressing 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Commit 5f3d439

rajumari-hub

 committed 19 minutes ago

build number 4

main

1 parent bde8318 commit 5f3d439

1 file changed

+1 -1

Filter files...

deployment.yaml

Search within code

deployment.yaml

+1 -1

@@ -23,7 +23,7 @@ spec:

23 23 containers:

24 24 - name: myapp

25 25

26 - image: 820982384863.dkr.ecr.us-east-1.amazonaws.com/myapp:3

26 + image: 820982384863.dkr.ecr.us-east-1.amazonaws.com/myapp:4

27 27

28 28 ports:

29 29 - containerPort: 80

Comments 0